

SQL MASTER GUIDE

From Database Structure to Advanced SQL

One complete, practical reference

DDL • DML • DQL • TCL • DCL • Statements • Clauses • Expressions • Functions • Aggregates • GROUP BY • HAVING • CASE • JOINS • Subqueries • CTEs • Window Functions • Transactions

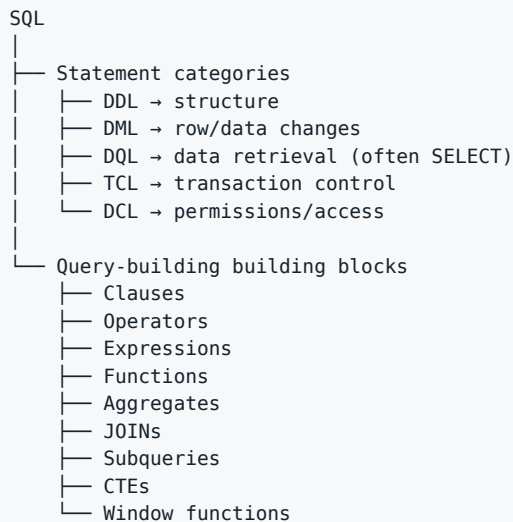
SQL	What kind of work?
DDL	Define / change database structure
DML	Insert / update / delete rows
DQL	Retrieve and analyze data
TCL	Control transactions
DCL	Control permissions
Query building	Clauses + expressions + functions + joins + subqueries + CTEs + windows

Learning principle: Do not memorize SQL as isolated commands. Understand what each layer does, then learn how the layers combine to answer real business questions.

1. The Big Picture: What SQL Really Is

SQL is a language used to communicate with relational database systems. A database has different jobs: creating structures, changing data, reading data, controlling transactions, and controlling access. That is why SQL statements are commonly grouped into categories.

The categories are logical classifications, not five physically separate languages. Different books and DBMS products may classify some statements differently. In particular, `SELECT` is sometimes treated as DQL and sometimes included under DML.



The key question: “What kind of job is this statement doing?” DDL asks what the database should look like. DML changes what is stored. DQL asks what data we want. TCL decides whether a set of changes is kept or undone. DCL decides who is allowed to do what.

A second key question is: “How is this query constructed?” That leads to clauses, expressions, functions, joins, subqueries, CTEs and window functions.

2. SQL Command Categories — Detailed

Category	Meaning	Main commands	Main purpose	School example
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define/change structure	Create students table; add email column
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Change rows/data	Add a student; change marks; remove a student
DQL	Data Query Language	SELECT	Retrieve/analyze data	Find students scoring > 80
TCL	Transaction Control Language	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT	Control a group of changes	Save or undo a multi-step update
DCL	Data Control Language	GRANT, REVOKE	Control permissions	Allow teacher to read student data

Memory trick: DDL builds the container; DML changes what is inside; DQL reads it; TCL controls a group of changes; DCL controls access.

Important distinction: DDL/DML/DQL/TCL/DCL describe statement purpose. Clauses, expressions, functions, JOINS, subqueries, CTEs and window functions describe the pieces and techniques used to construct powerful statements.

3. Build the Practice Database First

Use this school database throughout the guide. It is intentionally small enough to understand by hand, but relational enough to demonstrate joins, grouping, subqueries and advanced analysis.

```
CREATE DATABASE school;  
USE school;
```

The database has four related tables. The arrows show the foreign-key relationships:

```
departments  
  | department_id  
  ▼  
teachers  
  | teacher_id  
  ▼  
classes  
  | class_id  
  ▼  
students
```

3.1 Departments

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);  
  
INSERT INTO departments (department_id, department_name)  
VALUES  
(1, 'Mathematics'),  
(2, 'Science'),  
(3, 'English'),  
(4, 'Computer Science'),  
(5, 'Social Studies');
```

3.2 Teachers

```
CREATE TABLE teachers (  
    teacher_id INT PRIMARY KEY,  
    teacher_name VARCHAR(100),  
    department_id INT,  
    salary DECIMAL(10,2),  
    city VARCHAR(50),  
    FOREIGN KEY (department_id)  
        REFERENCES departments(department_id)  
);  
  
INSERT INTO teachers  
(teacher_id, teacher_name, department_id, salary, city)  
VALUES  
(101, 'Rajesh Kumar', 1, 65000, 'Hyderabad'),  
(102, 'Priya Sharma', 2, 72000, 'Hyderabad'),  
(103, 'Arun Reddy', 1, 68000, 'Warangal'),  
(104, 'Sneha Rao', 3, 60000, 'Vijayawada'),  
(105, 'Vikram Singh', 4, 80000, 'Hyderabad'),  
(106, 'Anita Reddy', 2, 75000, 'Warangal'),  
(107, 'Kiran Kumar', 5, 62000, 'Hyderabad'),  
(108, 'Meena Sharma', 4, 85000, 'Vijayawada');
```

3.3 Classes

```
CREATE TABLE classes (  
    class_id INT PRIMARY KEY,  
    class_name VARCHAR(20),  
    section VARCHAR(10),  
    teacher_id INT,  
    room_number INT,  
    FOREIGN KEY (teacher_id)  
        REFERENCES teachers(teacher_id)  
);  
  
INSERT INTO classes  
(class_id, class_name, section, teacher_id, room_number)  
VALUES  
(201, '8', 'A', 101, 101),  
(202, '8', 'B', 102, 102),  
(203, '9', 'A', 103, 201),  
(204, '9', 'B', 104, 202),  
(205, '10', 'A', 105, 301),  
(206, '10', 'B', 106, 302),  
(207, '11', 'A', 107, 401),  
(208, '11', 'B', 108, 402);
```

3.4 Students

```

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    class_id INT,
    gender VARCHAR(10),
    marks INT,
    city VARCHAR(50),
    FOREIGN KEY (class_id)
        REFERENCES classes(class_id)
);

INSERT INTO students
(student_id, student_name, class_id, gender, marks, city)
VALUES
(1, 'Aarav Sharma', 201, 'Boy', 85, 'Hyderabad'),
(2, 'Ananya Reddy', 201, 'Girl', 92, 'Hyderabad'),
(3, 'Rohan Kumar', 201, 'Boy', 68, 'Warangal'),
(4, 'Priya Singh', 201, 'Girl', 76, 'Hyderabad'),

(5, 'Arjun Rao', 202, 'Boy', 88, 'Vijayawada'),
(6, 'Sneha Reddy', 202, 'Girl', 95, 'Hyderabad'),
(7, 'Vivek Sharma', 202, 'Boy', 72, 'Warangal'),
(8, 'Kavya Rao', 202, 'Girl', 81, 'Vijayawada'),

(9, 'Rahul Kumar', 203, 'Boy', 65, 'Hyderabad'),
(10, 'Divya Singh', 203, 'Girl', 89, 'Warangal'),
(11, 'Aditya Reddy', 203, 'Boy', 91, 'Hyderabad'),
(12, 'Meena Rao', 203, 'Girl', 74, 'Vijayawada'),

(13, 'Ravi Sharma', 204, 'Boy', 78, 'Hyderabad'),
(14, 'Pooja Reddy', 204, 'Girl', 86, 'Warangal'),
(15, 'Karthik Rao', 204, 'Boy', 69, 'Vijayawada'),
(16, 'Isha Singh', 204, 'Girl', 93, 'Hyderabad'),

(17, 'Manish Kumar', 205, 'Boy', 91, 'Hyderabad'),
(18, 'Nisha Sharma', 205, 'Girl', 87, 'Warangal'),
(19, 'Rohit Reddy', 205, 'Boy', 76, 'Hyderabad'),
(20, 'Aisha Rao', 205, 'Girl', 95, 'Vijayawada'),

(21, 'Suresh Kumar', 206, 'Boy', 82, 'Warangal'),
(22, 'Neha Singh', 206, 'Girl', 90, 'Hyderabad'),
(23, 'Varun Rao', 206, 'Boy', 73, 'Vijayawada'),
(24, 'Kavya Sharma', 206, 'Girl', 88, 'Hyderabad'),

(25, 'Ajay Reddy', 207, 'Boy', 79, 'Hyderabad'),
(26, 'Sneha Kumar', 207, 'Girl', 94, 'Warangal'),
(27, 'Vishal Singh', 207, 'Boy', 85, 'Vijayawada'),
(28, 'Divya Rao', 207, 'Girl', 91, 'Hyderabad'),

(29, 'Rahul Sharma', 208, 'Boy', 88, 'Hyderabad'),
(30, 'Pallavi Reddy', 208, 'Girl', 96, 'Warangal'),
(31, 'Naveen Kumar', 208, 'Boy', 77, 'Hyderabad'),
(32, 'Anjali Singh', 208, 'Girl', 89, 'Vijayawada');

```

3.5 Schema reference

Table	Primary key	Foreign key	Relationship
departments	department_id	—	Parent of teachers
teachers	teacher_id	department_id → departments	Teacher belongs to department
classes	class_id	teacher_id → teachers	Class is taught by teacher
students	student_id	class_id → classes	Student belongs to class

4. DDL, DML and DQL in Practice

DDL changes the database design. DML changes rows. DQL retrieves rows. Keeping this distinction clear prevents many beginner mistakes.

DDL examples

```
-- Create
CREATE TABLE demo (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);

-- Change structure
ALTER TABLE demo
ADD city VARCHAR(50);

-- Remove all rows (DBMS behavior/transaction details vary)
TRUNCATE TABLE demo;

-- Remove the table object
DROP TABLE demo;
```

DML examples

```
-- Add a row
INSERT INTO students
(student_id, student_name, class_id, gender, marks, city)
VALUES
(33, 'Test Student', 201, 'Boy', 80, 'Hyderabad');

-- Change a row
UPDATE students
SET marks = 90
WHERE student_id = 33;

-- Delete a row
DELETE FROM students
WHERE student_id = 33;
```

DQL examples

```
SELECT student_name, marks
FROM students
WHERE marks >= 80
ORDER BY marks DESC;
```

DELETE vs TRUNCATE vs DROP

Operation	What it does	Typical use
DELETE	Removes selected rows; can use WHERE	Remove specific students
TRUNCATE	Removes all rows from a table	Empty a staging/table dataset quickly
DROP	Removes the table object itself	Remove an obsolete table

5. SQL Building Blocks: Statement → Clause → Expression

A statement is a complete instruction. A clause is a major piece of a statement. An expression produces a value. Functions operate on values. Operators combine or compare values.

```
SELECT class_id, AVG(marks)
FROM students
WHERE marks >= 50
GROUP BY class_id
HAVING AVG(marks) > 80
ORDER BY AVG(marks) DESC;
```

Layer	Meaning	Example
Statement	Complete instruction	SELECT ...;
Clause	Major query section	WHERE marks >= 50
Expression	Produces a value	marks + 5
Operator	Performs/composes comparisons or arithmetic	marks >= 80
Function	Calculates/transforms a value	UPPER(student_name)
Aggregate	Combines multiple rows into one value	AVG(marks)

Common clauses

Clause	Job
SELECT	Choose/output columns or expressions
FROM	Choose the source table/query
WHERE	Filter individual rows before grouping
GROUP BY	Create groups for aggregate calculations
HAVING	Filter groups after grouping
ORDER BY	Sort the final result
LIMIT / TOP / FETCH	Restrict returned rows; syntax depends on DBMS

6. Expressions — The Core of Powerful SQL

An expression is something SQL can evaluate to a value. This is the key idea behind combining SQL features.

```
marks + 5

UPPER(student_name)

CASE
  WHEN marks >= 80 THEN 'Good'
  ELSE 'Needs Improvement'
END
```

Inside-out reading rule: when an expression contains multiple layers, identify the innermost piece first, then move outward. For example, in `ROUND(AVG(CASE ... END), 2)`, first understand `CASE`, then `AVG`, then `ROUND`.

```
ROUND(
  AVG(
    CASE
      WHEN gender = 'Boy' THEN marks
    END
  ),
  2
)
```

Read it as: CASE produces a conditional mark value → AVG averages those values → ROUND formats the final average to 2 decimals.

7. Functions and Aggregate Functions

Scalar functions generally return one value for each input row. Aggregate functions combine values from multiple rows.

Type	Examples	Typical question
String	UPPER, LOWER, TRIM	How should names be cleaned/displayed?
Numeric	ROUND, ABS	How should numbers be transformed?
NULL handling	COALESCE, NULLIF	How should missing/special values be handled?
Aggregate	COUNT, SUM, AVG, MIN, MAX	What is the class average/count/total?

Nested functions are functions inside functions:

```
ROUND(AVG(marks), 2)

UPPER(TRIM(student_name))

ROUND(
  AVG(
    CASE WHEN gender = 'Boy' THEN marks END
  ),
  2
)
```

8. WHERE vs GROUP BY vs HAVING

WHERE filters rows. GROUP BY creates groups. HAVING filters groups.

```
SELECT
  class_id,
  AVG(marks) AS average_marks
FROM students
WHERE marks >= 50
GROUP BY class_id
HAVING AVG(marks) > 80;
```



```

WHERE marks >= 50
  ↓
filter individual students
  ↓
GROUP BY class_id
  ↓
create class groups
  ↓
AVG(marks)
  ↓
HAVING AVG(marks) > 80
  ↓
filter groups

```

Why not put AVG(marks) in WHERE? Because WHERE operates before grouping and the group average does not exist yet. HAVING is designed to filter after grouping.

9. CASE WHEN and Conditional Aggregation

CASE is an expression. It evaluates conditions and returns a value.

```

CASE
  WHEN marks >= 90 THEN 'Excellent'
  WHEN marks >= 75 THEN 'Good'
  ELSE 'Needs Improvement'
END AS performance

```

The real power appears when CASE is placed inside aggregate functions.

Pattern	Meaning
SUM(CASE WHEN condition THEN value ELSE 0 END)	Sum only rows meeting a condition
COUNT(CASE WHEN condition THEN 1 END)	Count rows meeting a condition
AVG(CASE WHEN condition THEN marks END)	Average only matching rows
MIN(CASE WHEN condition THEN marks END)	Minimum matching value
MAX(CASE WHEN condition THEN marks END)	Maximum matching value

Example: average marks for boys only

```

SELECT
  ROUND(
    AVG(
      CASE
        WHEN gender = 'Boy' THEN marks
      END
    ),
    2
  ) AS avg_boys_marks
FROM students;

```

Why does this work? For boys, CASE returns marks. For other rows, CASE returns NULL because there is no ELSE. AVG ignores NULL values, so only boys contribute.

Conditional counts

```

-- Portable pattern
COUNT(CASE WHEN gender = 'Boy' THEN 1 END)

-- PostgreSQL alternative
COUNT(*) FILTER (WHERE gender = 'Boy')

```

COUNT(expression) counts non-NULL expression results. That is why COUNT(CASE WHEN ... THEN 1 END) counts only matching rows.

10. Operators, DISTINCT, LIMIT and DESC

Feature	Example	Meaning
=	marks = 80	Equal
<>	marks <> 80	Not equal; standard SQL spelling
!=	marks != 80	Not equal; supported by many DBMS
>, <, >=, <=	marks >= 80	Comparison
AND / OR / NOT	marks >= 80 AND city = 'Hyderabad'	Combine conditions
IN	city IN ('Hyderabad','Warangal')	Match any listed value
BETWEEN	marks BETWEEN 70 AND 90	Inclusive range in common SQL usage
LIKE	student_name LIKE 'A%'	Pattern matching

DISTINCT removes duplicate result combinations:

```
SELECT DISTINCT city
FROM students;
```

LIMIT/TOP/FETCH syntax differs by DBMS. OFFSET skips rows and is commonly used for pagination:

```
SELECT *
FROM students
ORDER BY student_id
LIMIT 10 OFFSET 20;
```

DESC also has two meanings depending on context:

```
DESC students;           -- MySQL: describe table structure

SELECT *
FROM students
ORDER BY marks DESC;     -- descending sort
```

DESCRIBE/DESC is best understood as metadata/schema inspection, not as one of the five main command categories.

11. JOINS — Connecting Related Tables

The school schema is relational. JOINS let us bring related information together.

```

students
  | class_id
  ▼
classes
  | teacher_id
  ▼
teachers
  | department_id
  ▼
departments

```

The most important skill is identifying the relationship key. For students and classes, the correct condition is `s.class_id = c.class_id`, not a made-up same-name id relationship.

JOIN type	Keeps	Mental model
INNER JOIN	Only matching rows	Intersection
LEFT JOIN	All left rows + matches	Keep everything from left
RIGHT JOIN	All right rows + matches	Keep everything from right
FULL OUTER JOIN	All rows from both sides	Match where possible
CROSS JOIN	Every combination	Cartesian product
SELF JOIN	Same table used twice	A row related to another row in same table

11.1 INNER JOIN

```

SELECT
    s.student_name,
    c.class_name,
    c.section
FROM students AS s
INNER JOIN classes AS c
    ON s.class_id = c.class_id;

```

Only students whose class relationship matches are returned.

11.2 LEFT JOIN

```

SELECT
    c.class_name,
    c.section,
    s.student_name
FROM classes AS c
LEFT JOIN students AS s
    ON c.class_id = s.class_id;

```

Every class remains in the result. If a class has no student, student columns are NULL.

11.3 RIGHT JOIN

```

SELECT
    c.class_name,
    c.section,
    s.student_name
FROM students AS s
RIGHT JOIN classes AS c
    ON s.class_id = c.class_id;

```

RIGHT JOIN can often be rewritten as LEFT JOIN by swapping table order, so LEFT JOIN is commonly preferred for readability.

11.4 FULL OUTER JOIN

FULL OUTER JOIN returns all rows from both sides, matching where possible. MySQL does not natively provide FULL OUTER JOIN; a common simulation is LEFT JOIN UNION RIGHT JOIN.

```
SELECT s.student_name, c.class_name
FROM students s
LEFT JOIN classes c
    ON s.class_id = c.class_id
UNION
SELECT s.student_name, c.class_name
FROM students s
RIGHT JOIN classes c
    ON s.class_id = c.class_id;
```

11.5 CROSS JOIN

```
SELECT *
FROM teachers
CROSS JOIN departments;
```

With 8 teachers and 5 departments, the result has $8 \times 5 = 40$ combinations. There is deliberately no ON condition. Real uses include generating product $\times$ region grids or student $\times$ subject combinations.

11.6 SELF JOIN

```
SELECT
    e.employee_name AS employee,
    m.employee_name AS manager
FROM employees e
JOIN employees m
    ON e.manager_id = m.employee_id;
```

A self join uses the same table twice through aliases. It is useful for employee-manager hierarchies, parent-child categories, referrals, or comparing rows within one table.

12. Multi-Table JOIN: School Report

```
SELECT
    s.student_name,
    c.class_name,
    c.section,
    t.teacher_name,
    d.department_name
FROM students s
JOIN classes c
    ON s.class_id = c.class_id
JOIN teachers t
    ON c.teacher_id = t.teacher_id
JOIN departments d
    ON t.department_id = d.department_id;
```

Read the query as a chain: students connect to classes through class_id; classes connect to teachers through teacher_id; teachers connect to departments through department_id.

13. Subqueries — A Query Inside Another Query

A subquery is a query nested inside another statement. It is useful when one result becomes an input to another operation.

```
SELECT *
FROM students
WHERE marks > (
    SELECT AVG(marks)
    FROM students
);
```

Inside-out: first calculate the overall average; then return students whose marks are above that value.

Subqueries can appear in WHERE, FROM and other expression contexts, depending on the DBMS and syntax.

14. CTEs — Named Intermediate Results

A Common Table Expression (CTE) gives a subquery a name for the duration of one statement. It is often easier to read than deeply nested SQL.

```
WITH class_average AS (
    SELECT
        class_id,
        AVG(marks) AS avg_marks
    FROM students
    GROUP BY class_id
)
SELECT *
FROM class_average
WHERE avg_marks > 80;
```

Think: build an intermediate result → give it a name → query it.

CTEs are especially helpful when a problem has multiple logical steps or when the same intermediate result needs to be referenced more clearly.

15. Window Functions — Advanced Row-Aware Analysis

A window function calculates across a related set of rows while keeping the individual rows visible. This is different from GROUP BY, which collapses rows into groups.

```
SELECT
    student_name,
    marks,
    RANK() OVER (
        ORDER BY marks DESC
    ) AS ranking
FROM students;
```

A useful mental contrast:

Technique	What happens to detail rows?
GROUP BY + AVG	Many student rows become one row per group
Window AVG() OVER (...)	Student rows remain; each row can receive a group/overall calculation
RANK() OVER (...)	Student rows remain and receive a rank

Window functions are one of the major steps from intermediate to advanced SQL.

16. Transactions and ACID

A transaction is a logical unit of work. The goal is to keep related changes consistent: either the intended unit succeeds, or it can be rolled back as appropriate.

Property	Meaning	Bank-transfer intuition
Atomicity	All-or-nothing unit of work	Debit and credit belong to one logical operation
Consistency	Rules/invariants remain valid	Balances and constraints remain valid
Isolation	Concurrent transactions do not improperly interfere	One transaction should not see/produce an invalid intermediate state
Durability	Committed changes survive failures	After COMMIT, committed results should persist through a crash/reset

Basic transaction commands

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

If something goes wrong before completion, the transaction can be rolled back:

```
ROLLBACK;
```

SAVEPOINT allows a partial rollback point within a transaction:

```
START TRANSACTION;

UPDATE ...;

SAVEPOINT before_second_change;

UPDATE ...;

ROLLBACK TO SAVEPOINT before_second_change;

COMMIT;
```

Isolation is broader than one locking command. Common transaction isolation levels include READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ and SERIALIZABLE; exact behavior varies by DBMS.

SELECT ... FOR UPDATE is a practical locking tool used in transactional workflows:

```

START TRANSACTION;

SELECT balance
FROM accounts
WHERE account_id = 1
FOR UPDATE;

-- perform validated update(s)

COMMIT;

```

Memory: FOR UPDATE → think locking/isolation. COMMIT → finish the transaction and keep the changes. ROLLBACK → undo the current transaction's changes and finish it.

17. DCL — Permissions and Security

DCL answers: Who is allowed to do what?

```

GRANT SELECT
ON school.students
TO 'teacher_user';

REVOKE SELECT
ON school.students
FROM 'teacher_user';

```

Real systems commonly use roles and least-privilege access rather than giving every account unrestricted permissions.

18. The Most Important Skill: Combining SQL Pieces

Professional SQL is mostly about composition. You combine small, understandable pieces into a single statement.

```

SELECT
    c.class_name,
    c.section,
    ROUND(
        AVG(
            CASE
                WHEN s.gender = 'Boy' THEN s.marks
            END
        ),
        2
    ) AS avg_boys_marks
FROM classes c
JOIN students s
    ON c.class_id = s.class_id
WHERE s.marks >= 50
GROUP BY c.class_name, c.section
HAVING AVG(s.marks) > 70
ORDER BY avg_boys_marks DESC;

```

How to decode it:

- FROM + JOIN: build the combined row source.
- WHERE: remove rows that should not participate.
- CASE: turn each remaining row into a conditional value.
- AVG: combine those conditional values.

- ROUND: format the aggregate result.
- GROUP BY: define the class-level groups.
- HAVING: keep only groups meeting the aggregate condition.
- ORDER BY: sort the final groups.

This is the composition mindset: small operation → output value → feed that value into another operation → continue outward.

Useful combination patterns

```
-- Arithmetic + CASE
marks + CASE WHEN marks < 50 THEN 10 ELSE 0 END

-- String function + CASE
UPPER(
  CASE
    WHEN gender = 'Boy' THEN 'Male'
    ELSE 'Female'
  END
)

-- Aggregate + CASE
CASE
  WHEN AVG(marks) >= 80 THEN 'Excellent'
  WHEN AVG(marks) >= 60 THEN 'Good'
  ELSE 'Needs Improvement'
END

-- CASE inside aggregate
SUM(CASE WHEN gender = 'Boy' THEN marks ELSE 0 END)

-- Aggregate inside CASE
CASE
  WHEN AVG(marks) >= 80 THEN 'Excellent'
  ELSE 'Needs Improvement'
END
```

19. Beginner → Advanced → Professional Roadmap

Stage	Learn	Target ability
1. Foundation	SELECT, FROM, WHERE, operators	Retrieve and filter rows
2. Sorting/output	ORDER BY, DISTINCT, LIMIT/OFFSET	Control result shape
3. Expressions	Arithmetic, CASE, NULL, string/numeric functions	Create derived values
4. Aggregation	COUNT, SUM, AVG, MIN, MAX	Summarize data
5. Grouping	GROUP BY, HAVING	Analyze groups
6. Composition	Nested functions, CASE inside aggregates, conditional aggregation	Build layered logic
7. JOINS	INNER, LEFT, RIGHT, FULL concept, CROSS, SELF	Combine related tables
8. Subqueries	Scalar, correlated and derived-result patterns	Use one query's result in another
9. CTEs	WITH	Structure multi-step SQL
10. Window functions	RANK, ROW_NUMBER, SUM/AVG OVER	Analyze rows without losing detail
11. Transactions	ACID, COMMIT, ROLLBACK, SAVEPOINT, isolation	Write safe multi-step changes

Stage	Learn	Target ability
12. Performance	Indexes, EXPLAIN, query plans	Understand and improve performance
13. Professional and complex	Complex CTEs, windows, conditional metrics, data-quality	Complex logical reporting/analytics problems

20. Practice Questions Using the Same Database

Level	Question
Beginner	Find all students with marks ≥ 80 .
Beginner	List students from Hyderabad ordered by marks descending.
Beginner	Show distinct student cities.
Intermediate	Find the average marks of all students.
Intermediate	Find average marks by class_id.
Intermediate	Show only classes whose average marks exceed 80.
Intermediate	Count boys and girls using conditional aggregation.
Intermediate	Join students to classes and display class name/section.
Advanced	Join students $\rightarrow$ classes $\rightarrow$ teachers $\rightarrow$ departments and build a school report.
Advanced	Find students whose marks are above the overall average using a subquery.
Advanced	Build a CTE for class averages, then filter it.
Advanced	Rank all students by marks using RANK() OVER.
Advanced	Rank students within each class using PARTITION BY.
Professional	Create a report with class average, highest mark, pass count and top-ranked student.
Professional	Use EXPLAIN to inspect a query plan and reason about indexes.

21. One-Page Mental Cheat Sheet

Question	Think
What does this command do to structure?	DDL
What does this command do to rows?	DML
Am I retrieving data?	DQL / SELECT
Am I saving/undoing a unit of changes?	TCL
Who is allowed to do this?	DCL
Does this piece produce a value?	Expression
Does it calculate one row's value?	Scalar function/expression
Does it combine many rows?	Aggregate
Am I filtering rows?	WHERE
Am I creating groups?	GROUP BY

Question	Think
Am I filtering groups?	HAVING
Am I connecting tables?	JOIN
Do I need another query's result?	Subquery / CTE
Do I need to keep detail rows while calculating across them?	Window function
Do multiple changes need one logical unit?	Transaction / ACID

Final mental model



The goal is not to memorize a long list. The goal is to recognize the layer you need, choose the smallest operation that solves that part, and then compose those operations into a complete query.

Reference note: this guide follows the supplied learning material's terminology and examples, while also preserving the important caveat that SQL command categories vary by textbook and DBMS. The school schema and sample data are included so the concepts can be practiced as one connected system.